

Guía Maestra: Ecosistema de Desarrollo Autónomo con IA

Este documento define la arquitectura técnica, las herramientas activas y los flujos de trabajo operativos para un entorno de desarrollo asistido por IA. El objetivo es maximizar la autonomía de los agentes (Gentle AI / GSD) minimizando drásticamente el consumo de tokens y evitando la sobreingeniería o "colisión de prompts".

1. Arquitectura de Capas (Token Optimization)

El ecosistema se defiende del derroche de tokens mediante 4 barreras pasivas:

- **Capa 0: Higiene de Entrada (`.claudeignore`)**
 - **Función:** Bloquea el acceso de la IA a carpetas basura (`node_modules` , `__pycache__` , `.log` , `dist/`).
 - **Por qué:** Evita que el modelo Opus (caro) escanee miles de tokens inútiles durante la fase de *Planning* en GSD.
- **Capa 1: Filtro de Terminal (`RTK` - Rust Token Killer)**
 - **Función:** Proxy CLI que intercepta y comprime errores largos de compilación o tests.
 - **Por qué:** Evita que la IA lea *stack traces* kilométricos, dándole solo la línea que ha fallado.
- **Capa 2: Navegación Estructural (`LSP` **Nativo**)**
 - **Función:** Language Server Protocol activado en Claude Code.
 - **Por qué:** Sustituye al `grep` ciego. Permite a la IA saltar a definiciones y corregir errores de tipado en 50ms, con 100% de precisión.
- **Capa 3: Contingencia MCP (Desactivada por defecto)**
 - Solo se usará `RepoMapper` o `jCodeMunch` si nos enfrentamos a un monorepo masivo donde GSD se pierde, para darle un árbol AST del proyecto.

2. Los Dos Motores de Orquestación

Dependiendo de la envergadura de la tarea, se utiliza un orquestador u otro. Ambos se apoyan en **Engram** como disco duro de memoria a largo plazo.

A. Motor SDD: Gentle AI (Mantenimiento y Features Medias)

- **Arquitectura:** Delegación Estricta con Subagentes (Blank Slate).
- **Flujo Operativo:**
 1. **Lead Agent (Orquestador):** Lee este Canvas y el contexto de Engram. NO programa.
 2. **Subagente de Diseño (`sdd-specs`):** Nace sin contexto previo, lee el ticket, diseña la solución (usando Opus o Gemini 1.5 Pro) y la guarda en Engram. Se destruye.

3. **Subagente de Implementación (`sdd-apply`)**: Nace sin contexto, lee el diseño de Engram. Usa el **LSP (Capa 2)** para inyectar el código con modelo Sonnet 3.7. Se destruye.
4. **Subagente de Verificación**: Lanza tests. Si fallan, **RTK (Capa 1)** resume el error para que lo corrija rápido.

B. Motor GSD: Get Shit Done (Épicas y Migraciones Masivas)

- **Arquitectura**: Planner-Worker lineal.
- **Flujo Operativo**:
 1. **Planning (Opus)**: Analiza el repositorio respetando estrictamente el `.claudeignore` (Capa 0). Traza un plan de acción determinista.
 2. **Worker (Sonnet)**: Ejecuta el plan paso a paso usando el CLI y navegando por el código con el LSP (Capa 2).

3. Configuración Obligatoria del Entorno (Windows)

Para que este Canvas sea una realidad, el sistema local debe tener esto configurado:

1. Habilitar LSP en Claude Code: En el archivo `C:\Users\`

`<tu_usuario>\.claude\settings.json` :

```
{
  "env": {
    "ENABLE_LSP_TOOL": "1"
  }
}
```

2. Higiene de Proyecto (`.claudeignore`): En la raíz de todo proyecto tocado por GSD debe existir un `.claudeignore` :

```
node_modules/
__pycache__/
venv/
.env
*.log
dist/
build/
```

4. Lista Negra (Herramientas Prohibidas)

Para evitar la "Colisión de Prompts" (instrucciones contradictorias) y la sobreingeniería, las siguientes herramientas quedan explícitamente excluidas del stack:

- **✗ Claude-Pro-Optimizer (Reglas Globales / Skills)**: Descartado. Engram y SDD ya gestionan el flujo. Añadir más archivos globales confunde a los subagentes.

- **✗ Serena MCP:** Descartado. Redundante con el LSP nativo de Claude.
- **✗ Enjambres Conversacionales (CrewAI / AutoGen):** Descartados. Gastan tokens debatiendo. Preferimos la delegación estricta a través de Engram (Lectura/Escritura en disco)